

DATE	30 September 2026
TEAM ID	SWUID2026349965
PROJECT NAME	PocketSmart AI: Your Smart Budget & Recommendation Assistant

Phase 02: Requirement Analysis

1. Introduction

Requirement analysis defines what PocketSmart AI must provide to users and what the application must handle technically. The requirements are based on the three planners, authentication, the recommendation engine, the frontend pages and the history feature.

1.1 Stakeholders

Stakeholder	Interest
End user (shopper, host)	Quick, trustworthy budget plans
Developer team	Maintainable, modular code
Mentor and reviewers (SmartBridge, SmartInternz)	Correct use of GenAI, clean architecture, working demonstration
Shopping and service platforms	Receive visitors through search links

1.2 User Roles

Role	Permissions
Visitor	View the landing page, register and log in
Registered user	Use all three planners, view dashboard and history, log out

1.3 Functional Goals

- Collect budget and contextual information.
- Generate category-specific recommendations.
- Support Home, Party and Jewelry planning.
- Support registration, login, logout and protected sessions.
- Support optional image input for jewelry.
- Display recommendations and recommendation history.

2. Functional Requirements

ID	Requirement	Priority
FR-01	A user can register with username, email and password. Passwords are stored in hashed form.	High
FR-02	A user can log in and receive a secure access token kept in an HTTP-only cookie.	High
FR-03	A user can log out; the token is invalidated and the session is removed.	High
FR-04	The Home planner accepts total budget, number of lights, ceiling fans, furniture pieces and dining tables, selected rooms (Living Room, Kitchen, Bedroom) and extra notes.	High
FR-05	The Party planner accepts total budget, guest count, party type, venue type, needs (catering, decoration, entertainment) and extra notes.	High
FR-06	The Jewelry planner accepts total budget, occasion, style preferences and an optional outfit image.	High
FR-07	The system sends a structured request to Gemini 2.5 and receives a budget breakdown, items, estimated prices and search terms.	High
FR-08	The system generates shopping links for each item using platform lists suited to the category.	High
FR-09	The system calculates item count, total cost, percentage of budget per category and the remaining budget.	Medium
FR-10	Every plan is saved to history with time, inputs and the full result.	High
FR-11	A user can view the history list and open the full details of one recommendation.	Medium
FR-12	The dashboard shows a welcome message, planner shortcuts and recent activity.	Medium
FR-13	A clear message or fallback recommendation is shown if the AI response fails or is insufficient.	High

3. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	A recommendation is returned within about 15 seconds; pages load within 3 seconds.
NFR-02	Security	Passwords hashed with bcrypt; signed tokens with 30-minute expiry; API key stored only in an environment file.
NFR-03	Usability	Responsive forms and card layouts usable on phone, tablet and desktop, with clear labels.
NFR-04	Reliability	Errors are handled gracefully; the JSON part of the AI reply is extracted even if extra text surrounds it.
NFR-05	Scalability	Modular structure (routes, services, models) so new planners can be added easily.
NFR-06	Maintainability	Configuration through environment variables; prompt logic kept in one utility module.
NFR-07	Privacy	Users see only their own history; uploaded images are used only for analysis.
NFR-08	Compatibility	Works on current versions of Chrome, Edge, Firefox and Safari.

3.1 Input Validation Rules

- Budget must be a positive number.
- Guest count and item counts must be whole numbers, zero or more.
- Uploaded file must be an image (JPG, PNG or WEBP) within a size limit.
- Required fields cannot be blank; optional fields default to empty.

4. Technology, Data and Integration Requirements

4.1 Technology Stack

Layer	Technology	Purpose
Backend	Python 3.10+, FastAPI, Uvicorn	Routes, page serving, dependency handling
AI	Google Gemini 2.5 (Google Generative AI SDK)	Text and image reasoning, structured JSON output
Security	passlib (bcrypt), python-jose (JWT), OAuth2	Password hashing and access tokens
Frontend	HTML, CSS, JavaScript, Jinja2 templates	Forms and card-style result display
Utilities	Pillow, python-dotenv, Pydantic	Image handling, configuration, data schemas
Cloud reference	AWS CloudFront, Route 53, API Gateway, Lambda, S3, DynamoDB, Secrets Manager, IAM, CloudWatch	Reference architecture from the project brief

4.2 Data Requirements

Entity	Main fields
User	username, email, full name, hashed password
Session	username, login time, last activity, token, session data
Home input	total budget, lights, fans, furniture, dining tables, rooms, additional requirements
Party input	total budget, party type, guests, venue type, catering, decoration, entertainment, additional requirements
Jewelry input	total budget, occasion, preferences, outfit image path
Recommendation	id, username, timestamp, type, input summary, result summary, full result

4.3 Assumptions and Constraints

- A valid Gemini API key with enough quota is available.
- Prices are AI estimates for planning, not live listings.
- Shopping links are search links, so exact products depend on each platform.
- Users, sessions and history are held in memory in the demonstration build; a database is planned for production.
- An internet connection is required.

4.4 Risks and Mitigation

Risk	Mitigation
AI returns malformed output	JSON extraction helper and a fallback message
Total exceeds the budget	Strict prompt rules and backend calculation of remaining budget
API key exposure	Keep the key in the environment file; never commit it
Quota exhaustion	Clear error message and retry guidance